Kumi

The Coordination Backend

How many agents edit one repository at the same time without overwriting each other: purpose, architecture, and the reasoning behind each decision.

A technical reference for the services under `services/`. Drawn from the source tree: the coordinator, the persistence layer, the API gateway, the repository and integration services, and the shared type definitions they agree on.

1. What this system is for

Put two coding agents on one repository and they will produce two patches against the same file, and one of them will be lost. Put ten on it and the failure is no longer occasional; it is the normal outcome. Every mechanism described in this document exists to make concurrent agent work on a shared codebase safe, and to make the reasons for each decision legible afterwards.

The backend is not an agent and does not contain a model. It never writes code, never chooses an implementation, and on this deployment never even starts an agent process. What it does is narrower and, for a team, more valuable: it decides **who may touch what, in what order**, and it holds the single authoritative copy of the source that all of those agents are editing towards.

The three questions it answers

Question	Mechanism	Where it lives
May this agent start work at all, given what else is running?	Plan admission: a candidate plan scored against the set currently executing, then granted, reduced, sequenced or refused.	`coordinator/plan-admission.ts`
Is the work it came back with the work it was allowed to do?	Scope enforcement against the admitted plan, then exact-base integration onto canonical.	`coordinator/scope-validator.ts`, `integration-service`
What is the true state of the code right now?	The canonical repository: a server-side Git branch advanced only by compare-and-swap.	`repository-service`

> **The load-bearing idea**
>
> A plan is a promise about scope, made before any editing happens. Everything else follows from it. Arbitration is possible because plans can be compared; ownership is enforceable because a plan named what it wanted; a result is checkable because there is a declaration to check it against. An agent that edits without an admitted plan is refused at collection, not because of paperwork, but because no other holder ever had the chance to object to its scope.

What it deliberately does not do

- **It does not run agents.** With `COORD_LOCAL_AGENTS_ONLY=1`, every agent runs on a member's own machine under that person's own vendor subscription. This is a cost decision before it is anything else: RAM was roughly 70 percent of the hosting bill when execution was server-side.
- **It does not resolve merge conflicts by heuristic.** Conflicting work is ordered so that the conflict does not arise, or it is refused with the reason named.
- **It does not decide implementation.** A plan says which files an agent intends to touch. What it writes in them is the agent's business.

2. Service topology

The repository is a Turborepo monorepo. Seven services under `services/` make up the backend; four applications under `apps/` and three vendor adapters under `adapters/` sit around it.

Service	Responsibility
coordinator	The decision engine. Conflict detection, plan admission, ownership, scope validation, task decomposition, replay assessment, approvals. Roughly 28,000 lines, of which `coordinator.ts` is 6,137 and `plan-admission.ts` is 2,247.
persistence	Every durable fact. One store interface implemented three times (in-memory, SQLite, PostgreSQL) against a shared contract test suite, plus the migration ledger and the tamper-evident audit chain.
api-gateway	The only process reachable from outside. Authentication, authorization, routing, the channel and thread model, the remote worker protocol, the MCP endpoint, arbitration narration.
repository-service	Canonical Git. Bare repositories, canonical versions, bundles for remote workers, worktree creation, compare-and-swap advancement, push and pull to a remote.
integration-service	Applies an accepted changeset onto latest canonical in a temporary worktree, runs validation, commits the candidate, and advances the branch.
code-intelligence	Deterministic syntax indexing of a canonical revision. Feeds scope estimation, plan grounding and symbol-level arbitration. No model on the scheduling path.
workspace-manager	Git worktrees, Docker sandboxes where configured, changeset collection, and the AES sealing used for stored credentials and MCP secrets.

The dependency direction

```
api-gateway -> coordinator -> code-intelligence
| | |
+--------------+--------------------+--> persistence
|
+--> integration-service --> repository-service
|
workspace-manager <----------------+

shared-types is depended on by all of them and depends on none.
```

Nothing depends on the gateway. The coordinator has no knowledge of HTTP, channels, or users; it decides about tasks, plans and repositories, and the gateway is what turns those decisions into sentences in a room. That separation is why the same coordinator drives the in-process runner, a remote desktop worker, and an editor connected over MCP without knowing which is which.

3. The execution model

Understanding where code actually runs explains most of the architecture, including several things that would otherwise look like over-engineering.

Where	What runs there	Whose cost
Control plane (hosted)	The gateway, coordinator, persistence and canonical Git. Moves text, decides order, integrates patches. Never spawns an agent.	The deployment's, and it is small: no model inference, no agent memory.
A member's desktop	The worker process. Polls for work every five seconds, materialises a workspace, spawns the vendor CLI, returns a changeset.	That member's machine and their own vendor subscription.
A member's editor	Claude Code, Cursor or Codex, connected over MCP. Takes a task, does it in the checkout already open, reports a diff back.	The same, with no Kumi process running at all.

COORD_LOCAL_AGENTS_ONLY=1 is permanent on this deployment. It guarantees nothing runs on the control plane; it guarantees nothing about whose laptop.

The consequence worth internalising is that **the control plane is never the bottleneck and never the expense**. On a task that takes fourteen minutes, the backend's share is measured in seconds: a hardlinked local clone, a cached index, and a compare-and-swap. The rest is a model reading and writing code on somebody's own hardware.

4. The life of a task

A task moves through eight statuses. Seven of the transitions are ordinary; one, `open`, is the only non-terminal ending and exists for conversational work whose thread is waiting for the next message.

Status	Meaning	Leaves it when
`submitted`	Queued. Nothing holds it.	A worker, runner or editor leases it.
`claimed`	A lease exists. Evidence that it was taken, so a crashed run does not silently re-run.	The result is accepted, or the lease lapses and it returns to submitted.
`planned`	A plan was admitted and is waiting on a person before it may run.	An approval decision arrives, or the timeout expires.
`paused`	Held by an operator.	Resumed.
`open`	A conversational turn landed; the thread awaits a reply.	The next turn supersedes it, somebody ends it, or the silence outlasts the expiry sweep.
`integrated`	Landed on canonical, or answered a question with no patch to land.	Terminal.
`failed`	Tried and could not.	Terminal.
`cancelled`	Stopped before completion.	Terminal.

The path a successful task takes

```
1. intake objective estimated against the canonical index;
split into narrower tasks if it spans independent
modules and something else is competing for part

2. lease claimed atomically; base revision pinned

3. plan agent plans in its own worktree at that revision
(skipped entirely under a blanket claim)

4. admission scored against every plan currently executing;
granted / reduced / sequenced / refused

5. execute agent edits inside the admitted scope

6. collect changeset read off the worktree, checked against
the plan that was actually granted

7. integrate applied to a temp worktree at latest canonical,
validated, committed, compare-and-swapped in

8. settle leases released, workspaces removed, outcome
narrated into the thread
```

5. Canonical, plans and leases

Canonical

Canonical is a bare Git repository on the control plane, and it is the only authority on what the code is. It is advanced solely by compare-and-swap: an integration that finds the branch has moved underneath it leaves canonical untouched and the result is either replayed onto the newer revision or requeued to replan.

```
interface CanonicalVersion {
sequence: number; // monotonic, so 'newer' is a comparison
revision: string; // the commit
branch: string;
createdAt: string;
}
```

> **Canonical is not origin/main**
>
> The two diverge the moment anything integrates without being pushed. Pushing to a remote is an explicit action, not a consequence of integration. This is why a remote worker is handed a **bundle** rather than told to fetch: the bundle is the only channel through which a revision that exists solely in canonical can reach another machine.

Plans

An `AgentPlan` declares files, symbols, dependencies, APIs, schemas, configuration keys, tests, services, validation commands, external access, a risk level, and an optional statement of intent. The coordinator validates and normalises it before any execution workspace is granted.

Two fields deserve attention. `declared` keeps the agent's own words beside the widened lists that `enrichPlan` produces, because enrichment fills every list from the contents of the files a plan names, which is right for comparing two plans and wrong wherever a list is read as a claim. Without it a holder would claim every symbol in every file it mentioned, and unrelated plans would collide on naming conventions: any declaration ending in Schema, Entity, Model, Record, Payload, Input or Migration counted as a shared schema, at forty points, before a single file overlapped.

`claim` marks the two plans no agent wrote: a blanket claim over the whole repository, and the narrowed claim it becomes when somebody else arrives. Both are ordinary plans structurally, which is exactly what lets scope enforcement, ownership and arbitration keep working on a task that never planned.

Leases

A lease is the right to work on one task, held by one worker, pinned to one base revision. It is what makes concurrency safe at the outermost level: the claim and the lease are one transaction, so two workers polling at the same instant cannot receive the same task.

Field	Purpose
`baseRevision`	The canonical revision the workspace must be built from, and the base every hunk is later measured against.
`expiresAt` / `heartbeatAt`	Liveness. A worker renews on a timer; an editor renews by calling a tool. A lapsed lease returns its task to the queue.

Field	Purpose
`plan`	The admitted plan and the coordinator's answer, written under a compare-and-swap over the set of approved leases in that repository.
`claimableBy` (input)	Restricts a claim to work the caller may execute. A desktop offers exactly one person's vendor logins, so it may only take that person's tasks.
`repositoryParallelism`	How many active leases one repository may carry. Defaults to 1: strictly serialised remote execution.

Two window lengths, and the difference is not arbitrary. A desktop worker holds a lease for **five minutes** and renews it from a timer beside the process, so a short window only ever asks whether that process is alive. An editor holds one for **thirty minutes**, because it is renewed by an agent choosing to call a tool, and between two such calls a person can read a diff and come back.

6. Conflict evidence

`ConflictDetector` evaluates each plan pair and records transparent evidence. Every assessment stores its score, the resources involved, an explanation and a disposition, and the explanation names the structural subtotal the disposition was actually computed from.

Evidence	Relationship	Weight
`file_overlap`	Both tasks plan the same path	structural
`symbol_overlap`	Both change the same symbol	structural
`dependency_impact`	One produces a resource the other consumes	structural, directed
`api_impact`	API ownership or use overlaps	structural
`schema_impact`	Schema ownership or use overlaps	structural
`config_overlap`	Configuration keys overlap	structural
`test_overlap`	Test ownership or coverage overlaps	structural
`intent_conflict`	Stated intents judged to concern the same code	advisory only
`intent_independent`	Both intents resolved, nothing connects them	advisory, score zero

> **Why intent evidence can never block**
>
> Structural evidence can notify, sequence or block according to configurable weights. Intent evidence is scored and recorded and can raise a pair for a human to look at, but it is excluded from the total the thresholds are read against. This was only half true until 2026-07-31: the advisory score summed into the same total, and the guard applied only to pairs with no structural evidence at all. Measured against the team-queue runs the intent signal fired on four pairs and was wrong on all four, so the behaviour bought nothing but lost parallelism. The grounded assessor now shipping measured 70 percent precision against a bar of 80 that it did not clear; it is survivable only because of the advisory scoping, and `COORD_DISABLE_INTENT_GROUNDING=1` removes it.

7. Plan admission

The wave scheduler in the local coordinator decides from above: it holds every plan at once and assesses them pairwise before handing out a workspace. A remote worker has no such vantage point. It holds one plan and knows nothing about the others, so the same question has to be answerable from **one candidate plus the set currently executing**. That is what `PlanAdmissionController` is.

It re-implements nothing. The plan pair is scored by the same `ConflictDetector`, ownership is taken through the same `OwnershipService`, and both are evaluated from scratch on each call so an in-memory lease table cannot drift from the store's. The active-plan set reaches it through the leases each runner claims before publishing its plan.

The four answers

Outcome	What it means	What the agent does
approved	Nothing executing conflicts. Ownership granted over the declared resources.	Executes immediately.

Outcome	What it means	What the agent does
partial	Some of the plan is free. The contested resources are deferred and the reduced plan is what ownership was granted over.	Executes the granted part; the rest becomes a follow-up.
sequenced	It conflicts, and waiting behind the holder is the ordering answer.	Waits, then runs.
blocked	Refused. The holder must finish first.	Replans narrower, or is sequenced after repeated refusals.

The all-or-nothing answer is computed first, because that answer needs no qualification. Only a wholly refused plan is asked the finer question, and it is answered by re-deciding a *reduced* plan rather than waving the remainder through. Plans the index cannot vouch for, which is every plan in an empty repository, split on whole declared paths only: no line ranges and no symbol claims, because both are statements about contents this path cannot read.

Three constants that bound replanning

Constant	Value	What it does
`DEFAULT_PLAN_RETRY_MS`	15s	Spaces resubmission so a refusal is not a busy-wait.
`BLOCKED_ATTEMPTS_BEFORE_SEQUENCING`	2	Consecutive refusals spent on 'plan again, narrower' before sequencing instead. The first refusal is news; the second establishes that knowing did not help.
`BLOCKED_ADMISSION_LIFETIME_CAP`	4	Backstop for refusals spaced too far apart to form a run.

Both caps are liveness bounds, not safety valves. Each can only convert a refusal into a wait behind the holder, which is a stricter promise about ordering than the refusal was. Neither can admit anything.

8. The plan a solo task never writes

A plan exists so that somebody else can arbitrate against it. A task alone in its repository has nobody, so it is granted a **blanket claim**: the whole repository, recorded on its lease, with no agent call at all. It starts editing immediately. Nothing can conflict with it, because nothing else can be admitted while it is held.

This is the single largest saving available in the pipeline. It removes an entire agent turn, which on a large repository is a substantial fraction of wall-clock time, and it removes the planning tokens with it. In one ten-task live run, 74 percent of planning calls ended in a deferral at roughly 23,000 tokens each: the largest single line in the bill, almost all of it spent re-deriving plans already known to be unschedulable.

The conditions, all of which must hold

- Blanket claims are configured on (`COORD_BLANKET_CLAIM`, default on).
- The worker announced protocol version 3 or later. A worker that cannot be told its claim was narrowed must never be given one; it would hold the repository until its task ended.
- The lease carries no plan yet. Replacing an existing contract with a wider one is what the immutability rule on approved admissions forbids.
- **The objective anchored to at least one file.** An objective the estimator could not place cannot be narrowed by declaration either.
- No other lease is active in that repository.

> **The condition that catches people out**
>
> `estimateScope` reads the *objective text*, and its strongest signal is a literal path or directory: 'the strongest statement an objective can make about scope is to name the place'. An objective phrased visually, for instance 'remove this icon from the pinned sidebar' with a screenshot attached, anchors nothing. No estimate means no blanket claim, which means the full planning round trip is paid. Naming a file in the objective is the cheapest performance change available to a user of this system.

Narrowing a claim while it is held

When a second task is leased in that repository, the holder's coordinator notices (`watchBlanketClaim`) and freezes its claim to the files it has actually touched, read synchronously from its worktree at that moment, never from the periodic working-change poll which is allowed to lag by design. The narrowed claim is written under the same compare-and-swap every admission uses.

From then on the holder is an ordinary claimed task. Reaching outside its claim goes through the existing widening path, which grants if free and refuses immediately if taken, and a refusal at collection ends in a report naming the file and the holder rather than a silent escape. The freeze is to whole directories rather than individual files, so a wide refactor caught mid-sweep keeps moving.

9. Live scope negotiation and replanning

Widening mid-run

An executing agent may ask for additional files, symbols, APIs, schemas, configuration, tests or services. The coordinator rejects malformed or conflicting expansion, waits for a human where protected resources or risk require it, acquires new leases for accepted resources, persists the revised plan and the decision, and sends the answer back before the agent proceeds.

The final host-collected changeset must remain within the resulting approved plan. **An agent cannot make an undeclared patch acceptable by claiming it in a completion message.**

Replanning after canonical moves

Before a sequenced task executes, the coordinator compares its planning revision with current canonical. When canonical has moved: both revisions are indexed so additions, changes and removals are visible; a change notice is created for every supported resource class; the adapter receives its previous plan and a fresh planning worktree; the new plan is validated and conflict analysis recalculated; the revision and its reason are persisted before execution.

> **Replacement, not replay**
>
> This is plan replacement. The agent is responsible for adapting its implementation intent to the accepted producer contract, because a patch written against a file that has since changed is not automatically the right patch for the new one.

10. Integration

Accepted work never writes canonical directly.

```
1. freeze and collect the task overlay
2. check scope against the admitted plan, and approval policy
3. apply the patch to a temporary worktree at latest canonical
4. run configured validation, without a shell
5. commit the candidate with hooks disabled
6. compare-and-swap the canonical branch
7. persist the integration and canonical version records
8. release leases and remove temporary workspaces
```

A stale compare-and-swap or a failed command leaves canonical unchanged. Cleanup errors are recorded without rewriting an already-known integration outcome.

What happens when the base moved

Situation	Outcome
Canonical unchanged since the lease's base revision	Ordinary promotion.
Canonical moved, but nothing the result depends on changed	**Replay** onto the newer revision. The integration record keeps `replayedFrom` so history shows the result outlived its own base rather than hiding it.
Canonical moved in a way that matters	**Stale.** The task is requeued to replan. One re-assessment is budgeted (`STALE_REASSESSMENT_BUDGET = 1`); a result that loses twice is in a repository advancing faster than it can integrate, where replanning is both cheaper and likelier to be correct.
Some hunks conflict, the rest are clean	Optional **salvage**: the colliding patches are held back and returned as `salvagedDeferred` for the caller to requeue, rather than discarding every clean hunk with them.

Approvals are off by default

Step 2 consults the project's approval policy, and a project that has not configured one stops for nobody. As of 2026-08-06 the platform runs unattended by default: plans proceed on the coordinator's own evidence, and no human is asked.

That is a reversal. The previous default gated any plan declaring a schema change, touching a protected path, or rated high or critical, then waited up to 24 hours for a decision while the worker held its lease for up to eight. Unattended, that is a deadlock rather than a review: the first live benchmark gated 8 of 10 plans and spent its whole budget waiting for an approval nobody was there to give.

> **One exclusion, deliberately kept**
>
> Rolling canonical back still asks. `requireRollbackReview` defaults to true and is not governed by the `enabled` switch. Everything else here reviews work arriving through the pipeline; a rollback is the opposite operation. It discards changes that were already reviewed, validated and accepted, it is issued by an operator rather than produced by an agent, and nothing downstream re-checks it. A deployment that wants no prompt anywhere sets it to false and gets exactly that, but it has to say so.

Turning review back on is one field, per project:

```
{ "version": 1, "approvals": { "enabled": true } }
```

Nothing was removed from the approval machinery. Schema review, the protected-path list, the risk levels and the 24-hour timeout are all still there and are consulted the moment that switch is on. Only the answer to 'what happens when nobody has said' changed.

11. Narrating arbitration

An arbitration is the one thing this platform does that a pile of uncoordinated agents cannot, and for a long time it was invisible: the only symptom in the room was one agent mysteriously waiting. It is now announced twice, in two different voices, and the distinction is deliberate.

- **In the thread, the agent speaks for itself.** This task's own `plan_admitted` becomes a first-person line: waiting its turn, narrowing its plan, or starting on the part it was granted. A thread whose agent says 'working on it' and then goes silent is indistinguishable from a hang.
- **In the room, the coordinator speaks.** Neither agent decided the order, so putting the sentence in an agent's mouth would read as agents negotiating with each other. The line is always the same shape: who is involved, and the order they run in.

Situation	The room hears
sequence	A and B have conflicting files, B starts once A is done.
block	A and B have conflicting files, B will wait for A to go first.
partial admission	A and B have conflicting files, A starts on <granted files> now, <deferred files> once B is done.
both sides one agent	A is working on multiple tasks that conflict, it will do the other one first.
a hold clearing	A starts now, what it was waiting on is done.

Five rules that hold the shape in place

Each is a regression somebody reported.

- **Names, not prompts.** Both sides resolve through the same roster presentation every mention and message author uses. Matching is on the vendor, never on 'the first agent this person owns', which once produced a line naming the same agent on both sides.
- **One clause, not three.** The room wants the order, not the coordinator's reasoning.
- **No evidence dump.** The detector's full structural case goes to the audit record, which is where an argument that long belongs. Only a partial admission names files, because a split is illegible without them, and then at most four.
- **Silence where there is no decision.** A collision no admission acted on is not announced. Announcing it teaches readers to skim past the ones that do carry a decision.
- **One agent is not two.** One agent handed two colliding tasks is arbitrated identically, but the line names that agent once and tells the tasks apart by what each was asked to do.

12. The persistence layer

One interface, `CoordinationStore`, implemented three times: in-memory for tests and single-process use, SQLite for local and desktop deployments, and PostgreSQL for the hosted one. Roughly 15,700 lines of implementation behind a 2,900-line interface.

> **Why three implementations do not drift**
>
> `store-contract.test.ts` is a single suite run against all three backends. It asserts the things that actually diverge rather than the happy path: that a JSON column reads back as an array and not a string, that a boolean is a boolean and not 0, 1 or 'f', that ordering is stable, that a unique-constraint violation fails identically, and that a cascade removes join rows. A behaviour that only one store gets right is a bug the contract is supposed to catch before it ships.

Migrations

Schema changes are numbered, forward-only, and written twice: once for SQLite and once for PostgreSQL, with the PostgreSQL copy using `IF NOT EXISTS` guards. The pair carries the same version number and the same name, and `LATEST_SCHEMA_VERSION` is derived rather than declared. Migrations run on boot.

The audit chain

Every audit event carries the hash of its own payload and a chain hash folding in the previous event, so removing, reordering or editing any event breaks the chain from that point forward. Payloads are serialised with sorted keys, so the same content always hashes identically regardless of field order.

> **Detection, not prevention**
>
> An attacker with write access to the database can recompute the whole chain. What this buys is that a silent edit becomes expensive and an inconsistent history becomes obvious during review. It is stated this plainly in the source, because a tamper-evident log described as tamper-proof is worse than none.

Twenty-seven event types are recorded, spanning authentication, organisation and project changes, repository operations, and the whole task lifecycle: `task_submitted`, `task_decomposed`, `plan_received`, `plan_admitted`, `plan_revised`, `replan_requested`, `conflict_detected`, `ownership_granted`, `task_started`, `agent_progress`, `lease_expired` and the rest. The audit log is not incidental: it is where an argument too long for a channel goes, and it is what makes 'why did that task wait' answerable weeks later.

13. Reaching the backend from outside

Three clients drive the same coordinator through different transports. All of them are authenticated by the gateway, and none of them can reach the coordinator directly.

The remote worker protocol, version 4

```
lease -> bundle -> requestPlan -> POST .../plan -> execute -> result
|
+-- blocked / sequenced
-> wait or requeue
```

Version	What it added
2	Plan-first. A worker refuses to run against a control plane advertising version 1 rather than silently executing before its plan was arbitrated.
3	A claim on the whole repository can be narrowed while it is held, so a control plane only grants one to a worker announcing 3 or later.
4	An assignment may carry the project's approved MCP servers, secrets opened, for the one machine allowed to run them.

The floor a worker holds a control plane to is 3, not its own version: everything 4 added is optional, and refusing it would strand every desktop that updated before the server did.

Every worker endpoint requires the `run_task` scope, so a read-only token cannot pull work or return results.

The editor path

An editor connected over MCP does the work itself, with no Kumi process running on that machine. It shares the lease, the admission and the integration described above, and differs in exactly two respects.

- **The lease is taken up front; the plan is admitted at the end.** A worker declares its scope before it moves. An editor has already moved by the time the control plane hears from it, so the honest claim is the set of files its diff actually touched. That claim is strictly narrower than anything it could have promised in advance, and it is decidable at the moment it is made rather than guessed half an hour earlier. Admitting a blanket claim at take time would lock the repository for the whole window with a scope nobody could narrow, because there is no holder process to ask.
- **Liveness has three answers, not two.** A worker polls, so a task it can take starts within seconds. An editor cannot be woken and picks work up the next time somebody asks it to. Anything that tells a room work has *begun* has to know which it is talking to, or it reports a run that has not started.

14. Configuration reference

The environment variables that change coordination behaviour.

Variable	Effect
`COORD_LOCAL_AGENTS_ONLY`	Execution never happens on the control plane. Permanent on this deployment.
`COORD_BLANKET_CLAIM`	Blanket claims for solo tasks. Default on; `0` puts every task back through planning.
`COORD_REPOSITORY_PARALLELISM`	Active leases permitted per repository. Default 1.

Variable	Effect
COORD_DISABLE_INTENT_GROUNDING	Removes the grounded intent assessor, leaving the hardcoded verb-pair fallback.
COORD_DISABLE_PLAN_GROUNDING	Stops verifying plan declarations against the repository index.
COORD_STRICT_PLAN_REBASE	Tightens what counts as a plan surviving a canonical advance.
COORD_LEGACY_ADMISSION_LOOP	Restores the pre-deferral-ordering scheduling loop.
COORD_MCP_ENABLED	Managed MCP servers. A fence rather than a suggestion: off means nothing is attached or dialled, whatever is stored.
COORD_CREDENTIAL_KEY	The AES key sealing stored credentials and MCP secrets. Absent, those routes answer 501 rather than writing unrecoverable ciphertext.
COORD_DATABASE_URL	Selects the PostgreSQL store; absent, SQLite.
COORD_AUDIT_RETENTION_DAYS	How long audit events are kept in the live table before archiving. Zero keeps everything.
COORD_WORKER_ADAPTERS	What a worker advertises it can drive. Registration is the intersection of this and the project's configured agents.
COORD_WORKER_CONCURRENCY	Leases one worker may hold at once.

15. The invariants

If the rest of this document were lost, these are the properties the system is built to hold, and the ones any change should be checked against.

#	Invariant	Enforced by
1	Canonical advances only by compare-and-swap. A stale attempt leaves it untouched.	repository-service
2	No result is accepted without an admitted plan, because without one no other holder had the chance to object to its scope.	`acceptWorkResult`
3	A changeset must fall inside the plan that was *granted*, not the plan that was asked for.	scope validator, partial admission
4	Two workers polling simultaneously cannot receive the same task: the claim and the lease are one transaction.	`leaseNextTask`
5	An approved admission is immutable. No later request may widen or replace it.	`admitWorkPlan`
6	A lease that lapses returns its task to the queue rather than losing it.	expiry sweep
7	Refusal caps can only convert a refusal into a wait. Nothing in the liveness machinery can admit work.	admission constants
8	A worker may only take work belonging to the person who registered it, because it offers exactly one set of vendor logins.	`claimableBy`
9	A repository grant never widens into a project. Execution is narrowed to the repositories the caller actually holds.	gateway authorization
10	Nothing executes on the control plane while local-agents-only is set.	`COORD_LOCAL_AGENTS_ONLY`

Where the reasoning lives

This document is a summary. The source is the record, and it is written to be read: the comments in these services state what was tried, what broke, and what the measurement said, because a rule whose reason is not written down is a rule the next change will remove by accident.

Concern	File
Admission, partial admission, deferral	`services/coordinator/src/plan-admission.ts`
Conflict scoring and evidence	`services/coordinator/src/conflict-detector.ts`
Ownership modes and leases over resources	`services/coordinator/src/ownership-service.ts`
Blanket claims and their narrowing	`services/coordinator/src/blanket-claim.ts`, `blanket-holders.ts`
Scope estimation and decomposition	`services/coordinator/src/scope-estimation.ts`, `task-decomposition.ts`
Replay assessment after a canonical advance	`services/coordinator/src/replay.ts`
The store contract, and what it pins	`services/persistence/src/store.ts`, `store-contract.test.ts`
Audit hashing	`services/persistence/src/audit-chain.ts`
Worker protocol	`docs/protocol/remote-workers.md`

Concern	File
Editor path	`docs/protocol/editor-work.md`
This architecture, in the tree	`docs/architecture/coordination.md`